LinkedList

Puntatori a Puntatori

È possibile dichiarare puntatori a puntatori, che sono puntatori che memorizzano l'indirizzo di altri puntatori. Ecco un esempio:

```
int a = 10;
int *p;
int **pp;
p = &a;
pp = &p;
```

In questo esempio, `pp` è un puntatore a `p`, che a sua volta è un puntatore a `a`.

> 🎓 Le funzioni in C **ricevono parametri per valore**. Se passi un puntatore, la funzione riceve una copia del puntatore, quindi non può cambiarne l'indirizzo, solo i dati a cui punta.
> Per cambiare il **puntatore stesso**, serve un **puntatore a puntatore**.

CS50 LinkedList

https://youtu.be/0euvEdPwQnQ?t=1833&feature=shared

Definizione di un nodo

```
struct Nodo {
    int valore;
    struct Nodo* next;
};
```

Creazione di una lista

```
#include <stdio.h>
#include <stdlib.h>.
```

```c
struct Nodo {
    int valore;
    struct Nodo* next;
};

int main() {
    struct Nodo* head = malloc(sizeof(struct Nodo));
    head->valore = 10;
    head->next = NULL;

    printf("Valore nel primo nodo: %d\n", head->valore);

        // oppure Creazione lista vuota (secondo me è modal
ità migliore per iniziare)
        struct Nodo *lista = NULL; // lista vuota

    free(head);
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>.

struct Nodo {
    int valore;
    struct Nodo* next;
};

int main() {
    struct Nodo* head = malloc(sizeof(struct Nodo));
    head->valore = 10;
    head->next = NULL;

    printf("Valore nel primo nodo: %d\n", head->valore);

        // oppure (secondo me migliore)
        struct Nodo *lista = NULL; // lista vuota
```

```
    free(head);
    return 0;
}
```

Inserimento di nodi

Inserimento in testa

```
void inserisciInTesta(struct Nodo** head, int val) {
    struct Nodo* nuovo = malloc(sizeof(struct Nodo));
    nuovo->valore = val;
    nuovo->next = *head;
    *head = nuovo;
}
```

Inserimento in coda

```
void inserisciInCoda(struct Nodo** head, int val) {
    struct Nodo* nuovo = malloc(sizeof(struct Nodo));
    nuovo->valore = val;
    nuovo->next = NULL;

    if (*head == NULL) {
        // lista vuota → il nuovo nodo diventa la testa
        *head = nuovo;
    } else {
        struct Nodo* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = nuovo;
    }
}
```

Visualizzazione della lista

```c
void stampaLista(struct Nodo* head) {
    while (head != NULL) {
        printf("%d -> ", head->valore);
        head = head->next;
    }
    printf("NULL\n");
}
```

Ricerca ed eliminazione

```c
void eliminaNodo(struct Nodo** head, int val) {
    struct Nodo* temp = *head, *prev = NULL;
    while (temp != NULL && temp->valore != val) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return; // non trovato
    if (prev == NULL) *head = temp->next;
    else prev->next = temp->next;
    free(temp);
}
```

Esempio:

```c
#include <stdio.h>
#include <stdlib.h>

// Definizione del nodo
struct Nodo
{
    int valore;
    struct Nodo *next;
};

// Inserisce un nuovo nodo in testa
void inserisciInTesta(struct Nodo **head, int val)
```

```c
{
    struct Nodo *nuovo = malloc(sizeof(struct Nodo));
    nuovo->valore = val;
    nuovo->next = *head;
    *head = nuovo;
}

// Inserisce un nuovo nodo in coda
void inserisciInCoda(struct Nodo **head, int val)
{
    struct Nodo *nuovo = malloc(sizeof(struct Nodo));
    nuovo->valore = val;
    nuovo->next = NULL;

    if (*head == NULL)
    {
        *head = nuovo;
    }
    else
    {
        struct Nodo *temp = *head;
        while (temp->next != NULL)
        {
            temp = temp->next;
        }
        temp->next = nuovo;
    }
}

// Stampa tutta la lista
void stampaLista(struct Nodo *head)
{
    while (head != NULL)
    {
        printf("%d -> ", head->valore);
        head = head->next;
    }
    printf("NULL\n");
```

```c
}

// Elimina il primo nodo che contiene "val"
void eliminaNodo(struct Nodo **head, int val)
{
    struct Nodo *temp = *head;
    struct Nodo *prev = NULL;

    while (temp != NULL && temp->valore != val)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
        return; // non trovato

    if (prev == NULL)
    {
        // Il nodo da eliminare è la testa
        *head = temp->next;
    }
    else
    {
        prev->next = temp->next;
    }

    free(temp);
}

int main()
{
    struct Nodo *lista = NULL; // lista vuota

    // Lezione 3: inserimento
    inserisciInTesta(&lista, 10);
    inserisciInTesta(&lista, 20);
    inserisciInCoda(&lista, 5);
```

```c
    printf("Lista dopo inserimenti:\n");
    stampaLista(lista);

    eliminaNodo(&lista, 10);

    printf("Lista dopo eliminazione di 10:\n");
    stampaLista(lista);

    // liberare la memoria rimasta
    while (lista != NULL)
    {
        struct Nodo *temp = lista;
        lista = lista->next;
        free(temp);
    }

    return 0;
}
```

Visualization

Linked List Visualization | Log2Base2

Log2Base2 - The Visual Learning Platform
https://log2base2.com/dsa?utm_src=youtube&utm_target=l2b2ychannel

https://www.youtube.com/watch?v=hEl8FythuoY

